

CBT Examination Platform

Full-Stack Technical Architecture, Database Design, API Reference & Developer Manual

Author: Sudarshan Mishra | Version: 1.2.0 Production | Tech Stack: Next.js 16, React 19, Prisma 5, Supabase, NextAuth v5 | Target Scale: 1000+ concurrent users

1. EXECUTIVE SUMMARY & SYSTEM OVERVIEW

Product Vision: The Computer-Based Testing (CBT) Examination Software is a high-performance web platform designed to replicate national-level competitive exam environments such as SSC CGL, CHSL, IBPS PO/Clerk, Railways RRB, and State PSC exams. The platform delivers real-time time-bound examinations, color-coded question navigation, instant score computation with negative marking, and granular administrative controls.

Architecture Highlights: Built on the modern Next.js 16 App Router and React 19, the platform leverages PostgreSQL hosted on Supabase with transaction-level connection pooling (PgBouncer) to sustain simultaneous query traffic from 100+ examinees without database exhaustion. Security is enforced through NextAuth.js v5 with strict role-based access control (RBAC).

[!IMPORTANT NOTE] Mandatory Developer Directive â€ Branding Compliance

As per product guidelines, the branding string 'Designed & developed by Sudarshan Mishra.' is permanently rendered in bold typeface in both the top navigation header and bottom page footer across all views.

Component Layer	Technology	Key Configuration / Purpose
Framework	Next.js 16.3.5	App Router, Server Components, Route Handlers
Frontend UI	React 19 + Tailwind v4	SSC/IBPS exam layout, interactive palette
Database ORM	Prisma 5.14.0	Type-safe PostgreSQL schemas & query building
Database Engine	Supabase (PostgreSQL)	AWS Tokyo ap-northeast-1 with PgBouncer
Authentication	NextAuth.js v5 (Beta 25)	Google OAuth (Superadmin) + Credentials
Dev Engine	Webpack (--webpack)	Mandatory dev flag; bypasses Turbopack PostCSS bug
Hosting / CI/CD	Vercel + GitHub	Automated deployment pipeline on git push

2. ARCHITECTURE & HIGH CONCURRENCY DESIGN

Concurrency Strategy: Computer-based examinations experience burst traffic patterns: dozens or hundreds of candidates log in at the exact same minute, fetch all question payloads simultaneously, and submit responses in tight clusters. The system employs a dual-channel database connection architecture to guarantee zero connection timeouts.

â€ Transaction Pooler (Port 6543): Used by DATABASE_URL with ?pgbouncer=true. Handles all student read/write operations with connection multiplexing, supporting 100+ concurrent students on lightweight database compute.

- â **Direct Connection (Port 5432):** Used by DIRECT_URL for Prisma DDL migrations and admin schema updates (prisma db push) that require session-level locking unsupported by PgBouncer.
- â **Client-Side Exam State Isolation:** During the exam, candidate answers and review marks are held in React client state. Network traffic during the exam is zero until the final submit call, eliminating server strain during active test-taking.
- â **Zero-Downtime Deployment:** Prisma client generation and schema synchronizations execute during the Vercel build phase (npm run build), ensuring database tables are always pre-warmed before requests arrive.

PRISMA POOLER CONFIGURATION

```
// prisma/schema.prisma - High-Concurrency Connection Pooling
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")    // Pooler: port 6543 with ?pgbouncer=true
  directUrl = env("DIRECT_URL")     // Direct: port 5432 for migrations & DDL
}
```

[!WARNING] Turbopack macOS Known Incompatibility

Next.js 16.3 Turbopack crashes when parsing Tailwind CSS v4 on Apple Silicon machines. The development script in package.json is explicitly configured as 'next dev --webpack'. Developers must never revert this flag.

3. DATABASE SCHEMA & ENTITY RELATIONSHIP

Entity Architecture: The data layer is defined via Prisma Schema, modeling five core entities: Users, Exams, Questions, Responses, and Results.

Entity / Table	Key Attributes	Relational Constraints & Indexes
User	id, rollNo, name, email, password, role, dashboardAccess	rollNo (unique index), email (unique index), role: STUDENT ADMIN SUPERVISOR
Exam	id, title, duration, totalMarks, negativeMarks	Parent entity; 1-to-many with Question and Result
Question	id, examId, section, text, optionA-D, correctOption, marks, addedBy	belongsTo Exam; addedBy tracks faculty name
Response	id, userId, questionId, selectedOption, status	Composite unique [userId, questionId]; tracks review/answered flags
Result	id, userId, examId, score, correctCount, incorrectCount, completed	Composite unique [userId, examId]; locks student from re-taking

```
CORE PRISMA SCHEMA DEFINITIONS

model User {
  id          Int          @id @default(autoincrement())
  rollNo      String       @unique
  name        String
  email       String?      @unique
  password    String       @default("123456")
  role        Role         @default(STUDENT)
  dashboardAccess Boolean   @default(false)
  responses   Response[]
  results     Result[]
}

model Question {
  id          Int          @id @default(autoincrement())
  examId      Int
  exam        Exam         @relation(fields: [examId], references: [id])
  section     String
  text        String
  optionA     String
  optionB     String
  optionC     String
  optionD     String
  correctOption String    // 'A', 'B', 'C', or 'D'
  marks       Int          @default(1)
  addedBy     String?      // Faculty name audit trail
  createdAt   DateTime     @default(now())
  responses   Response[]
}
```

4. ROLE-BASED ACCESS CONTROL (RBAC) & SECURITY

Security Model: Security is governed by a strict three-tier hierarchy separating Superadmin, Admin (Faculty/Staff), and Students.

Operational Capability	STUDENT	ADMIN (Faculty)	SUPERADMIN (Owner)
Take Assigned Mock Examinations	YES	NO	NO
View Test Series Dashboard (/dashboard)	Paid Only	NO	YES
Create / Manage Exam Papers	NO	YES	YES
Add / Delete Questions (With Name Tag)	NO	YES (Own tag)	YES
Manage Students (Add, Delete, Reset Pwd)	NO	YES	YES
Toggle Student Dashboard Access (Paid)	NO	NO (Blocked)	YES (Sole Authority)
Permanently Delete Entire Exam Paper	NO	NO (Blocked)	YES (Sole Authority)
Grant / Revoke Admin Roles (/admin/users)	NO	NO (Blocked)	YES (Sole Authority)
Authentication Mechanism	Roll No + Pwd	Admin ID + Pwd + Name	Google OAuth 2.0 Only

Faculty Accountability & Attribution Architecture

Audit Logging: To prevent misuse in institutes with multiple teachers sharing the admin credential ('ADMIN' / 'adminpassword123'), the admin login screen (/admin-login) features a mandatory 'Your Full Name' input field. NextAuth binds this name to the active session token. When an admin creates a question, the API extracts session.user.name and writes it to the Question.addedBy database column. Question management cards prominently display 'Added by: [Faculty Name]'.

Superadmin Single-Sign-On Security

OAuth Whitelist: The Superadmin account uses Google OAuth restricted by email whitelist in src/auth.ts. Only logins matching 'sudarshan.contactwebdev@gmail.com' are granted access. Any other Google account is rejected.

5. EXAM ENGINE & CANDIDATE UI SPECIFICATION

Exam Delivery: The candidate examination interface (/exam/[id]) replicates official government CBT portals (TCS iON / SSC / IBPS).

â Question Palette States: Five distinct visual indicators: Green (Answered), Red (Not Answered), Purple (Marked for Review), Purple with Green Badge (Answered & Marked for Review), and Gray (Not Visited).

â Sectional Grouping: Questions automatically group into tabs (e.g., General Intelligence, Quantitative Aptitude, English, General Awareness) based on the section attribute.

â Countdown Timer: Synchronized in seconds from exam.duration. Renders in high-contrast monospaced font. Automatically submits the paper when timeLeft reaches 0.

â Anti-Cheat Result Locking: Once submitted, a Result record is created in the database. If a student attempts to log in again or refresh the page, the API detects the existing result and immediately renders the read-only Score Summary, preventing duplicate attempts or answer tampering.

â Question Key Security: The public exam API (/api/exam/[id]) strips the correctOption field from candidate responses. It is only included if an authenticated ADMIN or SUPERADMIN session is present.

6. DYNAMIC ROUTING & MONETIZED DASHBOARD

Routing Logic: The application provides seamless test routing while preserving a high-value commercial test series feature.

â Default Student Flow: When a student logs in at /login, the client calls /api/student/latest-exam, fetches the most recently uploaded exam ID, and navigates straight into /exam/[latestId]. Students always receive the current paper without administrative confusion.

â Monetized Test Library (/dashboard): The dashboard displays all past and upcoming test papers, duration, question count, and past performance scores. Access is locked behind the User.dashboardAccess flag.

â Superadmin Unlock Gate: In the /admin/students table, only the Superadmin sees the 'Dashboard' column. Clicking 'Locked' toggles the student's status to 'Granted'. Students without access see a clean locked screen prompting them to subscribe.

7. COMPLETE API ROUTE SPECIFICATION

API Directory: All endpoints are implemented as Next.js App Router Route Handlers located under src/app/api/.

Endpoint Route	HTTP	Access Level	Payload & Operational Behavior
/api/login	POST	Public	Body: {rollNo, password}. Returns User profile; sets local session.
/api/student/latest-exam	GET	Public / Student	Returns {examId: number} for the most recently created exam.
/api/student/exams	GET	Student	Query: ?userId=X. Returns list of all exams with candidate scores.
/api/student/dashboard-access	POST	Student	Body: {userId}. Checks if candidate has dashboardAccess enabled.
/api/exam/[id]	GET	Candidate / Admin	Returns exam metadata and questions. Strips correctOption for students.
/api/submit	POST	Candidate	Body: {userId, examId, responses}. Computes score and locks Result.
/api/admin/exam	GET, POST	ADMIN / SUPERADMIN	GET: list all exams. POST: creates exam {title, duration, marks, neg}.
/api/admin/exam/[id]	DELETE	SUPERADMIN ONLY	Cascades and deletes responses, results, questions, and the exam.
/api/admin/question	POST	ADMIN / SUPERADMIN	Creates question. Automatically binds session user name to addedBy.
/api/admin/question/[id]	DELETE	ADMIN / SUPERADMIN	Deletes question and associated student response rows.
/api/admin/students	GET, POST	ADMIN / SUPERADMIN	GET: list students. POST: registers new student {name, rollNo, pwd}.
/api/admin/students/[id]	DELETE, PATCH	Admin / Superadmin	DELETE: deletes student. PATCH: resets password or toggles dashboard.
/api/admin/users	GET, PATCH	SUPERADMIN ONLY	GET: all staff. PATCH: promotes or demotes user ADMIN role.

8. DIRECTORY STRUCTURE & CODEBASE MANIFEST

Comprehensive inventory of all critical project files and their architectural responsibilities:

- â **src/auth.ts:** NextAuth v5 configuration. Manages Google provider (Superadmin) and Credentials provider (Admin with fullName tracking). Emits JWT tokens containing role and name.
- â **src/app/layout.tsx:** Root layout wrapping SessionProvider. Enforces top and bottom bold branding: 'Designed & developed by Sudarshan Mishra.'
- â **src/app/page.tsx:** Root entrypoint. Automatically redirects visiting clients to /login.
- â **src/app/login/page.tsx:** Candidate login portal. Authenticates roll number and password, then routes to the latest active test paper.
- â **src/app/admin-login/page.tsx:** Isolated admin login page outside protected layouts (prevents redirect loops). Features Google Sign-In and Faculty Name input.
- â **src/app/admin/layout.tsx:** Server Component authentication gate. Inspects auth() session; redirects unauthorized clients to /admin-login.
- â **src/app/admin/page.tsx:** Main admin hub. Exam creation modal, exam listings, question manager links, and Superadmin delete button.
- â **src/app/admin/students/page.tsx:** Student registry. Searchable table with Delete, Reset Password, and Superadmin-only Dashboard Access toggle.
- â **src/app/admin/users/page.tsx:** Superadmin security console. Displays all users and controls ADMIN role promotion/revocation.
- â **src/app/admin/exam/[id]/page.tsx:** Question authoring interface. Form for 4 options, marks, correct key, and preview cards with 'Added by' audit badges.
- â **src/app/dashboard/page.tsx:** Candidate test library. Displays full suite of available exams; protected by per-student subscription gate.
- â **src/app/exam/[id]/page.tsx:** Full SSC/IBPS CBT engine. Split palette, live timer, response tracking, and read-only submission scorecard.
- â **prisma/seed.mjs:** Idempotent database seeding script. Provisions Superadmin, default shared Admin account, and sample mock exams.

9. DEVOPS, DEPLOYMENT & ENVIRONMENT SETUP

Deployment Workflow: The application is deployed on Vercel with automated CI/CD directly connected to the GitHub repository.

Environment Variable	Vercel Type	Description / Value Format
DATABASE_URL	Secret	Supabase PgBouncer pooler URL (Port 6543) with ?pgbouncer=true
DIRECT_URL	Secret	Supabase direct connection URL (Port 5432) for schema synchronization
GOOGLE_CLIENT_ID	Secret	OAuth 2.0 Web Client ID from Google Cloud Console
GOOGLE_CLIENT_SECRET	Secret	OAuth 2.0 Web Client Secret from Google Cloud Console
NEXTAUTH_SECRET	Secret	Cryptographic token string (e.g., cbt-exam-sudarshan-2026-secret)
NEXTAUTH_URL	Config	Canonical production URL (e.g., https://cbtfor-dsc.vercel.app)

Critical Build Script Optimization

Build Command: Vercel build settings run: 'prisma generate && prisma db push --accept-data-loss && node prisma/seed.mjs && next build'. The --accept-data-loss flag ensures seamless continuous schema pushes to PostgreSQL without stalling the deployment pipeline. Additionally, .npmrc contains 'legacy-peer-deps=true' to resolve NextAuth v5 beta peer dependencies cleanly.

Google Cloud Console OAuth Configuration

- â **Authorized JavaScript Origins:** https://cbtfor-dsc.vercel.app (Strictly origin only â no trailing slash, no path)
- â **Authorized Redirect URIs:** https://cbtfor-dsc.vercel.app/api/auth/callback/google (Exact callback route)

10. DEVELOPER ONBOARDING & SETUP INSTRUCTIONS

Local Setup Guide: Follow these exact steps to clone, configure, and launch the project on a local workstation:

```

TERMINAL COMMAND SEQUENCE

# 1. Clone repository from GitHub
git clone https://github.com/Sudarshan180392/cbtforDSC.git
cd cbtforDSC

# 2. Install dependencies (honoring .npmrc peer resolutions)
npm install

# 3. Configure local .env file (database URLs and auth secrets)
cp .env.example .env

# 4. Generate Prisma Client and synchronize schema with PostgreSQL
npx prisma generate
npx prisma db push --accept-data-loss

# 5. Run database seed script to populate Superadmin & default accounts
node prisma/seed.mjs

# 6. Launch development server using the mandatory Webpack engine
npm run dev      # Executes: next dev --webpack
```

[!TIP] Pre-Configured Default Credentials for Testing

Superadmin: sudarshan.contactwebdev@gmail.com (Google Sign-In) Staff Admin: Login ID:

'ADMIN' | Password: 'adminpassword123' | Full Name: Enter your name Demo Student: Roll No:

'SSC20260001' | Password: 'password123'